

Policy-Guided MPPI for Safe Manipulator Planning

Algorithm Implementation

February 11, 2026

1 Notation

- x_t : System state at time t
- u_t : Control input at time t
- $f(\cdot)$: System dynamics function
- H : Planning horizon
- K : Number of MPPI samples
- λ : Temperature parameter for MPPI
- Σ : Noise covariance matrix
- $\ell(\cdot)$: Stage cost function
- $\ell_f(\cdot)$: Terminal cost function
- π_ϕ : Learned policy with parameters ϕ
- $\mathcal{U}$: Control constraint set
- $\mathcal{F}_{\text{safe}}$: Safety filter function
- u_t^{safe} : Safe control input applied to the robot
- $\Pi_{\mathcal{U}}$: Projection onto control constraint set

2 Algorithm Description

The Policy-Guided MPPI (PG-MPPI) algorithm combines model predictive path integral control with a learned policy to enable safe and efficient manipulator planning. The algorithm operates in three main phases:

2.1 Offline Policy Training

First, a policy π_ϕ is trained in simulation using reinforcement learning methods such as SAC or CD-SAC. The training incorporates goal-reaching rewards and CDF-based safety shaping to ensure the policy learns both task completion and safety awareness.

2.2 Online Planning Loop

During execution, the algorithm performs the following steps at each time step:

1. **Policy-guided warm start:** The learned policy generates a nominal trajectory that serves as an initial guess for the MPPI optimization, improving convergence speed.
2. **MPPI sampling and update:** Multiple trajectories are sampled by adding noise to the nominal control sequence. Each trajectory is evaluated based on accumulated costs, and weights are computed using the exponential of negative costs. The control sequence is updated using a weighted sum of the noise terms.
3. **Safety filtering:** The first control action from the optimized sequence is passed through a safety filter that ensures collision-free motion and constraint satisfaction.
4. **Execution and receding horizon:** The safe control action is applied to the robot, and the planning horizon is shifted forward for the next iteration.

2.3 Key Features

- **Policy guidance:** The learned policy provides informed initialization, reducing the number of samples needed for good performance.
- **Safety guarantees:** The safety filter ensures that all applied control actions respect safety constraints, such as collision avoidance.
- **Receding horizon:** The algorithm continuously replans as new state information becomes available, enabling adaptation to disturbances and model uncertainties.

3 Applications

PG-MPPI is particularly suitable for:

- Robotic manipulator planning in cluttered environments
- Tasks requiring both goal-directed behavior and safety guarantees

- Situations where learned policies can provide good initial guesses but may not be fully reliable
- Real-time applications where computational efficiency is crucial

Algorithm 1 PG-MPPI (Policy-Guided MPPI) for Safe Manipulator Planning

Require: Dynamics $x_{t+1} = f(x_t, u_t)$; horizon H ; samples K ; temperature λ ; noise covariance Σ ; stage cost $\ell(x, u)$; terminal cost $\ell_f(x)$; learned policy $\pi_\phi(a | s)$ (e.g., SAC/CD-SAC actor); control constraint set $\mathcal{U}$; safety filter $\mathcal{F}_{\text{safe}}(\cdot)$ (collision-avoidance / constraint projection).

Ensure: Control u_t^{safe} applied to the robot at each time step.

```
1: (Offline, once) Train policy  $\pi_\phi$  in simulation (e.g., SAC/CD-SAC with
   goal reward + CDF-based safety shaping).
2: Initialize nominal control sequence  $U = \{u_0, \dots, u_{H-1}\}$  (zeros or from  $\pi_\phi$ ).
3: for  $t = 0, 1, 2, \dots$  do
4:   Observe current state  $x_t$ .
   Policy-guided nominal (warm start)
5:   Predict a nominal rollout under the learned policy:
6:    $\hat{x}_0 \leftarrow x_t$ 
7:   for  $i = 0$  to  $H - 1$  do
8:      $\hat{u}_i \leftarrow \mathbb{E}[\pi_\phi(\cdot | \hat{x}_i)]$  ▷ actor mean action
9:      $\hat{x}_{i+1} \leftarrow f(\hat{x}_i, \hat{u}_i)$ 
10:  end for
11:  Set (or blend) the nominal sequence  $U \leftarrow \{\hat{u}_0, \dots, \hat{u}_{H-1}\}$ .
   MPPI sampling and update
12:  for  $k = 1$  to  $K$  do
13:     $x_{k,0} \leftarrow x_t$ 
14:     $S_k \leftarrow 0$ 
15:    for  $i = 0$  to  $H - 1$  do
16:      Sample noise  $\epsilon_{k,i} \sim \mathcal{N}(0, \Sigma)$ 
17:       $u_{k,i} \leftarrow \Pi_{\mathcal{U}}(u_i + \epsilon_{k,i})$  ▷ clip/project to bounds
18:       $x_{k,i+1} \leftarrow f(x_{k,i}, u_{k,i})$ 
19:       $S_k \leftarrow S_k + \ell(x_{k,i}, u_{k,i})$ 
20:    end for
21:     $S_k \leftarrow S_k + \ell_f(x_{k,H})$ 
22:  end for
23:   $\beta \leftarrow \min_k S_k$  ▷ baseline for numerical stability
24:  for  $k = 1$  to  $K$  do
25:     $\tilde{w}_k \leftarrow \exp(-\frac{1}{\lambda}(S_k - \beta))$ 
26:  end for
27:   $w_k \leftarrow \tilde{w}_k / \sum_{j=1}^K \tilde{w}_j$  ▷ normalize
28:  for  $i = 0$  to  $H - 1$  do
29:     $u_i \leftarrow u_i + \sum_{k=1}^K w_k \epsilon_{k,i}$  ▷ path-integral update
30:     $u_i \leftarrow \Pi_{\mathcal{U}}(u_i)$ 
31:  end for
   Safety filtering and execution
32:    $u_t^* \leftarrow u_0$ 
33:    $u_t^{\text{safe}} \leftarrow \mathcal{F}_{\text{safe}}(x_t, u_t^*)$  ▷ e.g., collision-free projection/filter
34:   Apply  $u_t^{\text{safe}}$  to the robot, obtain next state  $x_{t+1}$ .
   Receding horizon shift
35:    $U \leftarrow \{u_1, u_2, \dots, u_{H-1}, u_{H-1}\}$  ▷ shift left and pad
36: end for
```
